

Process Creation

2026 Semester 2 COMPSCI 340: Operating Systems

Talia Xu

Lecture 4

2.1.1

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

Process Creation Turns a Program into Managed Execution

Recall: the previous lecture produced an **executable** by compiling and linking code with libraries.

An executable is still only a file. The operating system must create and manage a **process** before its instructions can run.

Today we will answer three questions:

1. What state makes up a process?
2. How does that process move between execution states?
3. How does Unix create a new process?

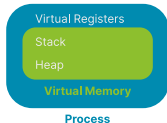
A Program Is Passive; a Process Is Active

Program

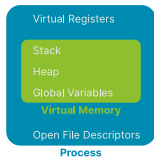
- Instructions and static data
- Stored in an executable file
- Has no current CPU state

One program can have zero, one, or many running instances.

A **process** is one running instance of a program plus the state needed to pause, protect, and resume it.



A Process Combines Execution, Memory, and Resources



Execution state

- Registers and program counter
- One or more thread stacks

Memory state

- Code, global data, heap, and mappings

Kernel-managed resources

- PID, open files, credentials, and limits

The PCB Is the Kernel's Record of a Process

A **process control block** (PCB) stores the information the kernel needs even while the process is not running.

Identity

- Process ID
- Parent ID
- Credentials

Execution

- Current state
- Saved registers
- Scheduling data

Resources

- Address space
- Open file table
- Accounting data

Creating a Process Means Constructing Its Initial State



The OS must:

- allocate kernel bookkeeping and an address space,
- establish initial resources such as standard input, output, and error,
- initialize registers so execution begins at the program entry point, and
- place the new process in the scheduler's ready queue.

Processes Share the CPU but Keep Separate Address Spaces

CPU virtualization

- Many processes appear to run at once
- The scheduler chooses a runnable process
- Context switches save and restore registers

Memory isolation

- Each process uses its own virtual addresses
- One process cannot directly access another's memory
- Sharing requires an explicit OS mechanism

A Thread Is an Execution Stream Inside a Process

Program

- File containing code and data

Process

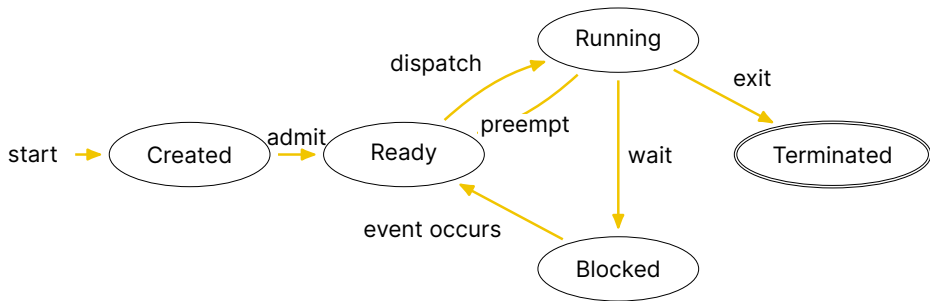
- Address space
- Open resources
- At least one thread

Thread

- Program counter
- Registers
- Private stack

Threads in one process share code, global data, the heap, and open resources, but each thread has its own execution state.

Five States Describe a Process Lifecycle



One I/O Request Explains the State Transitions

Follow a process that reads a file:

1. **Created** → **Ready**: the OS finishes initialization.
2. **Ready** → **Running**: the scheduler dispatches it.
3. **Running** → **Blocked**: a read must wait for storage.
4. **Blocked** → **Ready**: the device reports completion.
5. **Ready** → **Running**: the process is scheduled again.
6. **Running** → **Terminated**: it finishes or exits.

A blocked process is not competing for CPU time until its event occurs.

Linux Exposes Process Identity and State

From a program

```
pid_t me = getpid();  
pid_t parent = getppid();
```

A PID identifies one process; a PPID records its parent process.

From the shell

```
$ ps -o pid,ppid,state,cmd  
$ cat /proc/1234/status
```

/proc presents kernel process information through a file-like interface.

These tools show the abstractions represented by the PCB without exposing the kernel data structure itself.

Operating Systems Expose Different Creation Interfaces

Win32 CreateProcess

- Names the program to load
- Creates a process and initial thread
- Configures handles and environment

POSIX fork + exec

- fork: clone the current process
- Modify the child if needed
- exec: replace its program

fork Creates a Child from the Current Process

```
pid_t fork(void);
```

The return value tells the two execution paths apart:

Return value	Meaning
-1	Creation failed; there is no child
0	This code is running in the new child
> 0	This code is running in the parent; value is the child's PID

After a successful call, parent and child continue from the instruction after fork.

Trace Both Paths After fork

```
int main(void) {  
    if (fork() == 0)  
        printf("Child\n");  
    else  
        printf("Parent\n");  
    return 0;  
}
```

Scheduling Makes the Output Order Nondeterministic

The child receives 0 and prints Child; the parent receives a positive child PID and prints Parent. Either process may run first:

Parent runs first

Parent
Child

Child runs first

Child
Parent

fork creates concurrency; it does not promise which process runs next.

If order matters, the program needs synchronization such as the parent's wait operation.

Trace value in the Parent and Child

```
int value = 2;
pid_t result = fork();

if (result == 0)
    value = 3;

printf("%s: %d\n",
       result == 0 ? "child" : "parent",
       value);
```

Parent and Child Have Separate Values

One possible order

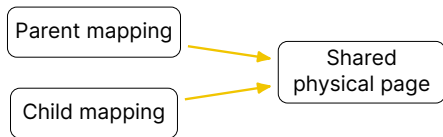
child: 3
parent: 2

- The child changes its own copy to 3
- The parent's copy remains 2
- Scheduling may reverse the two output lines

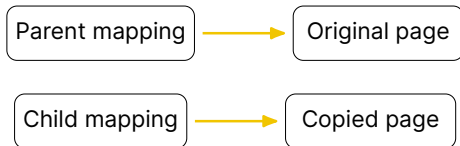
A child write does not modify the parent's address space.

Copy-on-Write Makes fork Cheaper

Immediately after fork



After the child writes



Initially, parent and child can map the same physical pages. A write triggers a private copy for the writing process.

The address spaces are logically separate even while unchanged pages are physically shared.

Trace Two Consecutive fork Calls

```
fork();  
fork();  
printf("hello\n");
```

Two fork Calls Create Four Execution Paths

- Start with 1 process
- First fork: $1 \rightarrow 2$
- Both processes call the second fork: $2 \rightarrow 4$
- All four processes reach printf

execve Replaces the Program, Not the Process

Replaced

- Code and global data
- Heap and stack
- Register context
- Program arguments and environment

Preserved

- Process ID and parent relationship
- Many process attributes
- Open file descriptors unless marked close-on-exec

A successful `execve` begins the new program at its entry point and never returns to the old program.

execve Names the Program, Arguments, and Environment

```
int execve(const char *path,  
           char *const argv[],  
           char *const envp[]);
```

- path: executable file to load
- argv: null-terminated array of argument strings; argv[0] conventionally names the program
- envp: null-terminated array of environment strings
- Return value: -1 on failure; no return on success

Trace the Code After execve

```
int main(void) {  
    printf("I'm going to become another process\n");  
    char *args[] = {"ls", NULL};  
    char *env[] = {NULL};  
  
    int result = execve("/usr/bin/ls", args, env);  
    if (result == -1) {  
        int error = errno;  
        perror("execve failed");  
        return error;  
    }  
    printf("This line is unreachable\n");  
}
```

Successful execve Does Not Return

Success

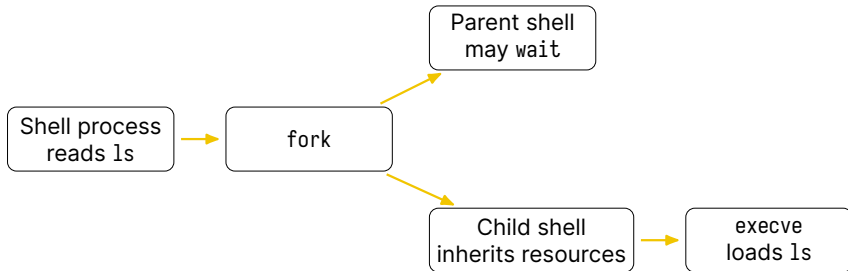
- The process begins running `/usr/bin/lis`
- The old code and stack are replaced
- The line after `execve` is never reached

Failure

- `execve` returns `-1`
- `perror` reports the reason
- The old program can recover or exit

A return value is needed only for failure because success replaces the caller.

A Shell Uses fork Then exec



The child can redirect file descriptors after fork and before exec; the new program then inherits that setup.

Trace the Lifecycle of `ls` from the Shell

1. The running shell reads the command `ls`.
2. `fork` creates a child with a new PID and copied process state.
3. The child prepares resources, then `execve` replaces its address space with `ls`.
4. The scheduler runs `ls`; it may block while reading files or writing output.
5. `ls` exits; the waiting parent collects its status and prompts for another command.

`fork` creates the process relationship; `exec` selects the new program; scheduling and waiting coordinate their execution.

Process Identity Explains the Results

- fork creates a separate child process with a different PID
- exec changes the program but preserves the process PID
- Successful execve replaces its caller and does not return
- The scheduler, not source order, decides which process prints first

Summary: Unix Separates Creation from Program Loading

- A process combines CPU state, an address space, and kernel-managed resources
- The PCB lets the kernel identify, pause, schedule, and resume that process
- Process states distinguish ready-to-run work from blocked work
- fork creates a child with separate logical state, implemented efficiently with copy-on-write
- execve replaces the program while keeping the process identity
- Shells compose fork, resource setup, exec, and wait

Key distinction: fork answers “which process?”, while exec answers “which program runs in it?”